

# **XENIX<sup>®</sup> System V/286 Operating System**

---

**User's Guide**

Information in this document is subject to change without notice and does not represent a commitment on the part of Microsoft Corporation. The software described in this document is furnished under a license agreement or nondisclosure agreement. The software may be used or copied only in accordance with the terms of the agreement. It is against the law to copy this software on magnetic tape, disk, or any other medium for any purpose other than the purchaser's personal use.

© Copyright Microsoft Corporation, 1985

Microsoft, the Microsoft logo and XENIX are registered trademarks of Microsoft Corporation.

The High Performance Software is a trademark of Microsoft Corporation.

Document Number: 290190003-500-R00-0785

# Contents

---

## 1 Introduction

- 1.1 Overview 1-1
- 1.2 The XENIX System 1-1
- 1.3 The XENIX Working Environment 1-1
- 1.4 Using This Manual 1-2

## 2 Demonstration

- 2.1 Introduction 2-1
- 2.2 Before You Use XENIX 2-1
- 2.3 Logging In 2-1
- 2.4 Typing Commands 2-2
- 2.5 Correcting Typing Errors 2-4
- 2.6 Read-Ahead and Type-Ahead 2-4
- 2.7 Strange Terminal Behavior 2-4
- 2.8 Stopping a Program 2-5
- 2.9 Logging Out 2-5
- 2.10 Learning More 2-5

## 3 Basic Concepts

- 3.1 Introduction 3-1
- 3.2 Files 3-1
- 3.3 File Systems 3-3
- 3.4 Naming Conventions 3-3
- 3.5 Commands 3-7
- 3.6 Input and Output 3-8

## 4 Tasks

- 4.1 Introduction 4-1
- 4.2 Gaining Access to the System 4-1
- 4.3 Configuring Your Terminal 4-3
- 4.4 Editing the Command Line 4-4
- 4.5 Manipulating Files 4-4
- 4.6 Manipulating Directories 4-10
- 4.7 Moving in the File System 4-13
- 4.8 Using File and Directory Permissions 4-14
- 4.9 Processing Information 4-18
- 4.10 Controlling Processes 4-22
- 4.11 Getting Status Information 4-24
- 4.12 Using the Lineprinter 4-26

- 4.13 Communicating with Other Users 4-29
- 4.14 Using the System Clock and Calendar 4-30
- 4.15 Using the Automatic Reminder Service 4-31
- 4.16 Using Another User's Account 4-31
- 4.17 Calculating 4-31

## **5 Vi: A Text Editor**

- 5.1 Introduction 5-1
- 5.2 Demonstration 5-1
- 5.3 Editing Tasks 5-15
- 5.4 Solving Common Problems 5-45
- 5.5 Setting Up Your Environment 5-46
- 5.6 Command Summary 5-51

## **6 Mail**

- 6.1 Introduction 6-1
- 6.2 Demonstration 6-1
- 6.3 Basic Concepts 6-4
- 6.4 Using Mail 6-8
- 6.5 Commands 6-12
- 6.6 Leaving Compose Mode Temporarily 6-20
- 6.7 Setting Up Your Environment: The .mailrc File 6-24
- 6.8 Using Advanced Features 6-27
- 6.9 Quick Reference 6-29

## **7 The Shell**

- 7.1 Introduction 7-1
- 7.2 Basic Concepts 7-1
- 7.3 Redirecting Input and Output 7-5
- 7.4 Shell Variables 7-10
- 7.5 The Shell State 7-14
- 7.6 A Command's Environment 7-16
- 7.7 Invoking the Shell 7-17
- 7.8 Passing Arguments to Shell Procedures 7-17
- 7.9 Controlling the Flow of Control 7-19
- 7.10 Special Shell Commands 7-30
- 7.11 Creation and Organization of Shell Procedures 7-32
- 7.12 More About Execution Flags 7-33
- 7.13 Supporting Commands and Features 7-34
- 7.14 Effective and Efficient Shell Programming 7-40
- 7.15 Shell Procedure Examples 7-44
- 7.15 Shell Grammar 7-52

## **8 BC: A Calculator**

- 8.1 Introduction 8-1
- 8.2 Demonstration 8-1
- 8.3 Tasks 8-3
- 8.4 Language Reference 8-14

## **A Ed**

- A.1 Introduction A-1
- A.2 Demonstration A-1
- A.3 Basic Concepts A-2
- A.4 Tasks A-3
- A.5 Context and Regular Expressions A-22
- A.6 Speeding Up Editing A-35
- A.7 Cutting and Pasting with the Editor A-39
- A.8 Editing Scripts A-41
- A.9 Summary of Commands A-42





# Chapter 1

## Introduction

---

- 1.1 Overview 1-1
- 1.2 The XENIX System 1-1
- 1.3 The XENIX Working Environment 1-1
-  1.4 Using This Manual 1-2





## 1.1 Overview



This manual introduces the XENIX® system by explaining the fundamental concepts and software needed to use it effectively. The XENIX system is an improved and enhanced version of the UNIX® SYSTEM V from AT&T. It is intended for use in schools, corporations, laboratories and small office environments. The system is well known as a productive environment for software development and has been used for many years as a text processing environment.

## 1.2 The XENIX System

The XENIX system consists of a general-purpose multiuser operating system and over one hundred utilities and application programs. In addition to the XENIX Timesharing System described in this manual, two other XENIX system packages are available: the XENIX Software Development System and the XENIX Text Processing System.

## 1.3 The XENIX Working Environment



The XENIX system is built around the XENIX operating system, which organizes and controls computer resources such as memory, disks, lineprinters, terminals, and any other peripheral devices connected to the system. XENIX is a *multiuser* and *multitasking* operating system: a multiuser system permits several users to use a computer simultaneously, thus providing lower cost in computing power per user; a multitasking system permits several programs to run at the same time, thereby increasing productivity.

Because UNIX (and thus XENIX) has been accepted as a standard for high-end operating systems, a great deal of software is available for this environment. In addition, XENIX is a bridge to the MS-DOS® operating system, the most widely used 16-bit operating system in the world. For systems that support MS-DOS, XENIX provides commands that let you access MS-DOS format files and disks.

Other characteristics of the XENIX system include

- A powerful command language for programming XENIX commands. Unlike other interactive command languages, the XENIX *shell* is a full programming language.
- Simple and consistent naming conventions. Names can be used absolutely, or relative to any directory in the file system.
- Device-independent input and output: each physical device, from interactive terminals to main memory, is treated like a file, allowing uniform file and device input and output.

- A set of related text editors, including a full screen editor.
- Flexible text processing facilities. XENIX commands let you find and extract patterns of text from files, compare and find differences between files, and search through and compare directories. Text formatting, typesetting, and spelling error-detection facilities, as well as a facility for formatting and typesetting complex tables and equations, are also available.
- A sophisticated calculator program.
- Mountable and dismountable file systems that permit addition of floppy disks to the file system.
- Flexible directory and file protections that provide the owner of each file or directory (or groups of users) with complete read, write, and execute access.
- Facilities for creating, accessing, moving, and processing files and directories in a simple and uniform way.

The XENIX system also includes enhancements developed at the University of California at Berkeley, and a visual interface similar to other Microsoft® productivity tool interfaces.

## 1.4 Using This Manual

| See                         | To                                                                                                                      |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Chapter 1, "Introduction"   | become acquainted with the XENIX system.                                                                                |
| Chapter 2, "Demonstration"  | gain hands-on experience with the XENIX system.                                                                         |
| Chapter 3, "Basic Concepts" | become acquainted with fundamental XENIX concepts: the file system, naming conventions, commands, and input and output. |
| Chapter 4, "Tasks"          | learn to use XENIX commands to perform everyday tasks.                                                                  |

|                                |                                                                                     |
|--------------------------------|-------------------------------------------------------------------------------------|
| Chapter 5, "Vi: A Text Editor" | learn to use the screen editor, <b>vi</b> .                                         |
| Chapter 6, "Mail"              | learn to use the XENIX mail facility.                                               |
| Chapter 7, "The Shell"         | become acquainted with the shell command interpreter and learn to write procedures. |
| Chapter 8, "BC: A Calculator"  | learn to use BC, a sophisticated calculator program.                                |
| Appendix A, "Ed"               | to learn to use the editor, <b>ed</b> .                                             |

You can obtain additional information on the XENIX operating system from the following manuals:

#### *Introducing XENIX*

This manual introduces the novice computer user to the XENIX operating system.

#### *Installation/Operation/User's Guide*

The *Installation Guide* explains how to install the XENIX system on a 286(80286)-based computer. It is written for the user who is not familiar with XENIX. The *Operations Guide* is also intended for the inexperienced user. It explains how to operate and maintain the XENIX operating system to ensure maximum performance. The *User's Guide* introduces the XENIX system by explaining the fundamental concepts and how to use the system and utilities effectively.

#### *Visual Shell User's Guide*

This manual explains how to use Microsoft's visual shell. Screen images and step-by-step instructions assist the inexperienced user. Menu customization and addition of new commands are covered for the advanced user.

#### *XENIX Reference Manual*

This manual is a two-volume set intended for all XENIX system users. It is a complete reference guide to XENIX commands, utilities, system calls, and file formats. The first volume contains timesharing and miscellaneous commands. The second volume contains the programming commands, the system calls, and file formats.

*Programmer's Guide*

This guide explains how to use the XENIX Software Development System to create and maintain Microsoft C Compiler and assembly language programs. It explains how to use the system resources that are available to the programmer in the XENIX environment.

*C User's Guide/C Library Guide*

The *C User's Guide* helps the beginning C programmer use the XENIX tools to design, compile, link, and debug C language programs. The *C Library Guide* provides information about the standard include files, describes how to build user interfaces for C programs, and provides a list of library routines common to both MS-DOS and XENIX.

*C Language Reference*

This manual defines the C language as implemented by Microsoft. It is designed as a language reference for programmers with experience in C or another programming language.

*Macro Assembler User's Guide/Macro Assembler Reference Manual*

The *Macro Assembler User's Guide* helps the beginning assembly language programmer use the XENIX tools to design, compile, link, and debug assembly language programs. The *Macro Assembler Reference Manual* is designed as a language reference for experienced programmers writing assembly language programs.

*Text Processing Guide*

This manual is intended for anyone using the text editing, formatting, and word processing tools provided in the Text Processing System. The interrelationships of the various tools are described in detail with examples of their use.



# Chapter 2

## Demonstration

---

- 2.1 Introduction 2-1
  - 2.2 Before You Use XENIX 2-1
  - 2.3 Logging In 2-1
  -  2.4 Typing Commands 2-2
  - 2.5 Correcting Typing Errors 2-4
  - 2.6 Read-Ahead and Type-Ahead 2-4
  - 2.7 Strange Terminal Behavior 2-4
  - 2.8 Stopping a Program 2-5
  - 2.9 Logging Out 2-5
  - 2.10 Learning More 2-5
- 



## 2.1 Introduction

The demonstration in this chapter presents the basic concepts of the XENIX system. You will learn how to

- log in
- type at your keyboard
- correct typing errors
- enter commands
- log out

## 2.2 Before You Use XENIX

Before you can use the system, your name must appear on the XENIX user list. See your system administrator, or refer to the *XENIX Operations Guide* for detailed information on adding users to the system.

Once your name is on the XENIX user list, you have a XENIX *account*. With this account you'll be given a user name, a password, and a login directory. Once you have these, all you need is a terminal from which you can enter, or *log into* the system. XENIX supports most terminals; see the *XENIX Operations Guide* for information on how to configure your terminal.

## 2.3 Logging In

The “login:” prompt should appear on the terminal screen.

To login

1. Type your login name.
2. Press RETURN.

If a password is required, you will be asked for it.

3. Type the password.
4. Press RETURN.

The password does not appear on the screen as you type; this prevents others from viewing it.

A successful login produces a *prompt character*, a single character that indicates the system is ready to accept commands. The prompt is usually a dollar sign (\$) or a percent sign (%). You may also receive a login message such as

```
you have mail
```

indicating that another system user has sent you mail. Refer to Section 4.13.2 to find out how to read your mail.

---

### Note

If the system displays nonsense characters when you type, your terminal is probably receiving information at the wrong speed. Check your terminal switches with the `stty` command. (See `stty(C)` in the *XENIX Reference Manual* for information on setting terminal switches.) If the switches are set correctly, but the problem still occurs, press the INTERRUPT key a few times.

---

## 2.4 Typing Commands

Once the prompt character appears, the system is ready to respond to commands typed at the terminal. Try typing

```
date
```

followed by RETURN. The system responds with a line of the form

```
Mon Jun 16 14:17:10 EST 1983
```

Remember to press the RETURN key at the end of each command line. On some terminals RETURN may be labeled ENTER or CR, but in all cases the key performs the same function.

Another command you might try is **who**, which lists the names of everyone who is logged in to XENIX. A typical display from the **who** command might look something like this:

```
you      console Jan 16   14:00
joe      tty01   Jan 16   09:11
arn      tty02   Jan 16   09:33
```

The time, given in the fourth column, indicates when the user logged in; `tty nn` is the system name for each user's terminal, where `nn` is a unique two-digit



number. “Console” is the special name of the system master terminal.

If you make a mistake typing the command name, you will see a message on your screen. For example, if you type

```
whom
```

the system responds with the message

```
whom: not found
```

Note that case is significant in XENIX. The commands

```
who
```

and

```
WHO
```

are not the same.

Now try displaying a message on your screen using the **echo** command. Type

```
echo hello world
```

The **echo** command echoes the rest of the command line to your terminal:

```
hello world
```

Now try this:

```
echo hello world >greeting.file
```

This time the **echo** command sends its output to a new file named *greeting.file* instead of to your terminal. Note the use of the greater-than sign (>) to *redirect* the output of the command from your screen to a file.

Now type

```
ls
```

to list just the name of the file, *greeting.file*. To look at the contents of *greeting.file*, type

```
cat greeting.file
```

“Cat” stands for concatenate. In this instance, the **cat** command sends the following output to your terminal screen:

```
hello world
```

To remove *greeting.file*, type

```
rm greeting.file
```

XENIX command names are often shortened to mnemonic names. For example, **ls** is short for “list,” **cat** is short for “concatenate,” and **rm** is short for “remove.”

## 2.5 Correcting Typing Errors

Provided you have not pressed RETURN, there are two ways to correct typing errors on a command line:

- Press the BKSP to erase the last character typed.

You can use the BKSP key to erase characters back to the beginning of the line, but not beyond.

- Press CTRL-U to erase all characters on the current input line.

## 2.6 Read-Ahead and Type-Ahead

XENIX has full read-ahead, which means that XENIX remembers what you have typed, no matter how quickly or when you type. If you enter characters while a command is displaying text on the screen, XENIX stores and interprets the characters in the correct order, even though they appear to be intermixed with the screen text. Full read-ahead lets you type several commands (i.e., “type ahead”) one after another without waiting for the first to finish execution. Note that this doesn't work when you log in; type-ahead works only *after* you have entered your password and received the prompt.

## 2.7 Strange Terminal Behavior

Occasionally, your terminal may act strangely. To fix the problem, try resetting the terminal with the following actions:

- Turn your terminal off, then quickly back on.
- Log out and log back in.

If these procedures don't solve the problem, read the description of the *stty(C)* command in the *XENIX Reference Manual* for more information about setting terminal characteristics.

## 2.8 Stopping a Program

To stop the execution of most programs and commands, press the INTERRUPT (DEL) key. (Refer to the hardware manual for your computer if you do not have an INTERRUPT or DELETE key.) Inside some programs, like most text editors, typing INTERRUPT stops whatever the program is doing without aborting the program itself. Throughout this manual, the phrase “send an interrupt” means to press the INTERRUPT key.

## 2.9 Logging Out

To end a XENIX session, you must log out:

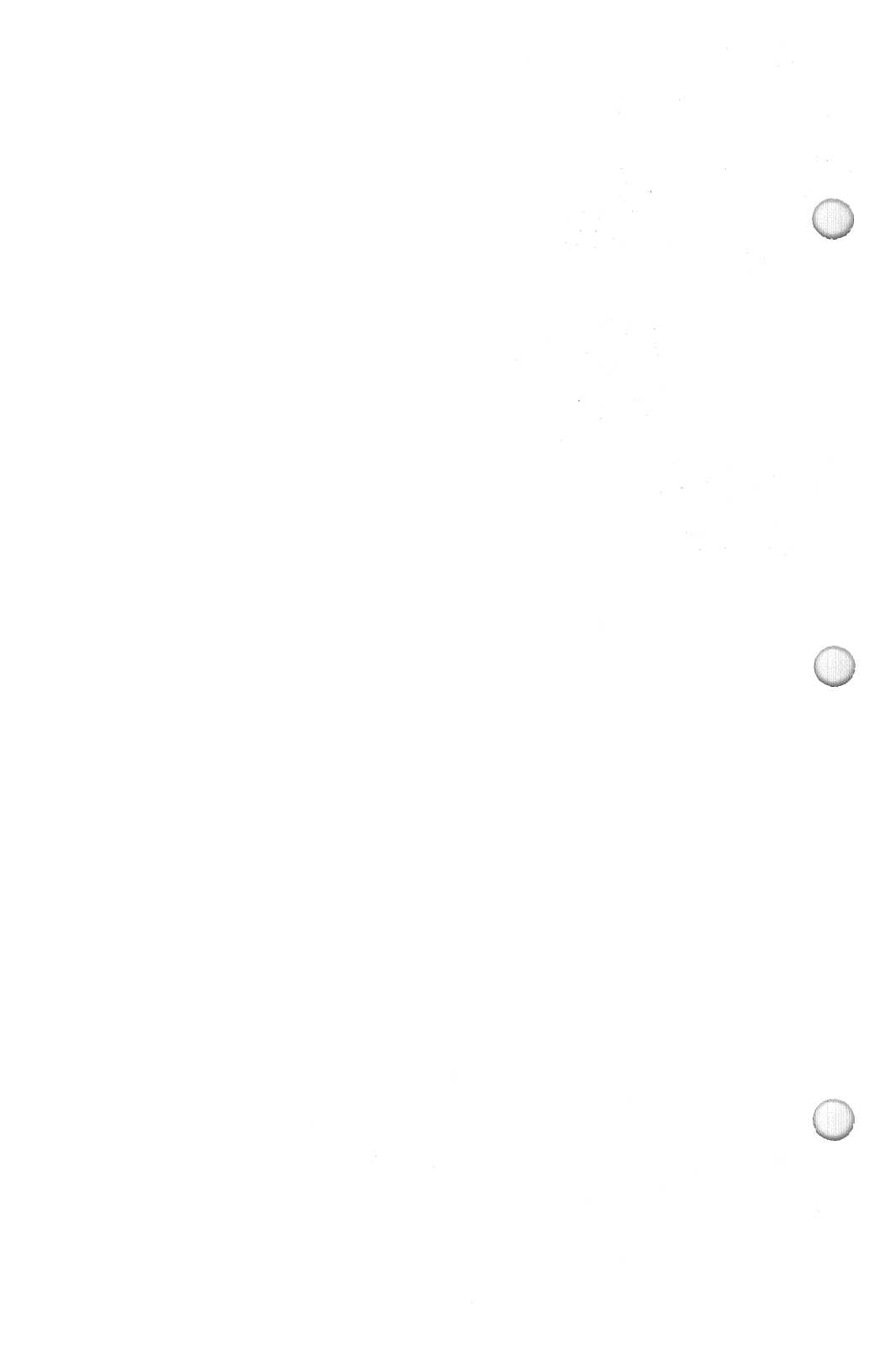
1. Move to a new line.
2. Press CTRL-D.

It is not sufficient just to turn off the terminal, since this does not log you out. CTRL-D, also ends some programs, so beware.

## 2.10 Learning More

The skills you have learned in this demonstration will get you started using XENIX. You will probably want to learn how to use the XENIX shell and a text editor. You use the shell language to speak to the system and execute commands. Text editors let you create and edit files, documents, and programs. The **vi** editor and the shell are discussed thoroughly in Chapters 5 and 7, respectively. The **ed** editor is discussed in Appendix A.

The next two chapters introduce you to basic XENIX concepts and commands. Be sure to read these chapters; it is especially important to understand how the file system is organized and how files are named.



# Chapter 3

## Basic Concepts

---

- 3.1 Introduction 3-1
- 3.2 Files 3-1
  - 3.2.1 Ordinary Files 3-1
  - 3.2.2 Directory Files 3-2
  - 3.2.3 Special Files 3-2
  - 3.2.4 Directory Structure 3-2
- 3.3 File Systems 3-3
- 3.4 Naming Conventions 3-3
  - 3.4.1 Filenames 3-3
  - 3.4.2 Pathnames 3-4
  - 3.4.3 Sample Names 3-4
  - 3.4.4 Special Characters 3-5
- 3.5 Commands 3-7
  - 3.5.1 Command Line 3-7
  - 3.5.2 Syntax 3-8
- 3.6 Input and Output 3-8
  - 3.6.1 Redirection 3-9
  - 3.6.2 Pipes 3-10



## 3.1 Introduction

This chapter introduces you to the basic concepts you need to function in the XENIX environment. After reading this chapter you should understand how the system's files, directories, and devices are organized and named, how commands are entered, and how a command's input and output can be manipulated.

## 3.2 Files

The file is the fundamental unit of the XENIX file system. In XENIX there are really three different types of files:

- Ordinary files (what we usually mean when we say “file”)
- Directories
- Special files

### 3.2.1 Ordinary Files

An ordinary file is simply a named concatenation of 8-bit bytes. Whether these bytes are interpreted as text characters, binary instructions, or program statements is up to the programs that examine them. Ordinary files typically contain textual information such as documents, data, or program sources. Executable binary files are also of this type.

Every ordinary file has the following attributes:

- A filename (not necessarily unique)
- A unique system number (*inode number*)
- A size in bytes
- A time of creation
- A time of modification
- A time of last access
- A set of access permissions

Access permissions insure file privacy and security. The owner provides read-write-execute permissions to files to control access by the owner, by a group of users, and by anyone else. By default, the owner of a file is its creator. The owner can read the file or write to it. By default, other users can read a file owned by another, but not write to it. File permissions can be altered with the **chmod** command, described in Section 4.8.1.

### 3.2.2 Directory Files

Directory files are read-only files containing the name and *inode number* of each file or directory residing within the given directory. An inode number is a unique number associated with a given file. *All* files on the system have inode numbers. A name/inode number pair is called a *link*. You use the **ls** command to examine directory files and to list file information. When paired with the inode number, the **ls** command can find additional information about a file.

Like ordinary files, directories can be by assigned appropriate access permissions to insure privacy and security. By default, the owner of a directory can read, create or remove files within that directory. Similarly by default, a user can read files within the directory of another, but not add or remove them. See Section 4.8 for detailed information on access permissions.

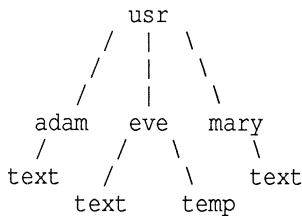
### 3.2.3 Special Files

Special files correspond to physical devices such as hard and floppy disks, lineprinters, terminals, and system memory. They are called *device special files*, and are not discussed in this manual.

### 3.2.4 Directory Structure

XENIX organizes all files into a structured directory hierarchy. Each system user has his own personal directory, commonly known as his *home directory*. A home directory may contain subdirectories over which the user has ownership and control.

A diagram of part of a typical user directory is shown in Figure 3-1.





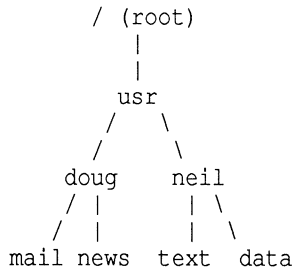
**Figure 3-1** A Typical User Directory

In Figure 3-1, the *usr* directory contains each user's personal directory. Notice that Mary's file named *text* is unrelated to Eve's. When you log in to XENIX, you automatically enter the system at your home directory. Any files you create will initially reside here.

### 3.3 File Systems

A file system is a set of organized files. In XENIX, this set of files consists of all available resources including data files, directories, programs, line-printers, and disks. Thus, the XENIX file system is a system for accessing all system resources.

The XENIX file system is organized hierarchically. Figure 3-2 illustrates a typical file system.

**Figure 3-2** A Typical File System

- The root directory, */*, is the highest-level directory
- *U**sr*, *d**ou**g*, and *n**ei**l* represent directories
- *M**a**i**l*, *n**e**w**s*, *t**e**x**t*, and *d**a**t**a* represent data files.

### 3.4 Naming Conventions

Every file, directory, and device in XENIX has both a filename and an absolute pathname. The pathname is a map of the file or directory's location in the system. The absolute pathname is unique within the entire system; filenames are unique only within directories and need not be unique system-wide.

#### 3.4.1 Filenames

A simple filename is a sequence of one to fourteen characters. Every file, directory, and device in the system has a filename. Filenames uniquely identify directory contents. While no two filenames in a directory may be the

same, filenames in *different* directories may be identical.

Although you can use almost any character in a filename, it is best to confine filenames to the alphanumeric characters and the period. The following characters have special XENIX functions and should not be used in filenames:

- Dashes (-)
- Question marks (?)
- Asterisks (\*)
- Brackets ([ ])
- All quotation marks ( ' “ ” ’ )
- Slashes (/) and backslashes (\)
- Control characters

### 3.4.2 Pathnames

A pathname is a sequence of directory names followed by a simple filename. Elements of the pathname are separated by a slash:

`/usr/doug/news`

A pathname beginning with a slash is called a *full pathname*: it specifies a file that can be found by beginning a search at the *root* directory. A pathname without an initial slash is called a *relative pathname*: it specifies files that can be found by beginning the search at the user's *current directory* (also known as the *working directory*).

### 3.4.3 Sample Names

Some sample names follow:

|                   |                                                                                                                                                                     |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/</code>    | The absolute pathname of the root directory of the entire file system.                                                                                              |
| <code>/bin</code> | The directory containing most of the frequently used XENIX commands.                                                                                                |
| <code>/usr</code> | The directory containing each user's personal directory. The subdirectory, <code>/usr/bin</code> contains frequently used XENIX commands not in <code>/bin</code> . |
| <code>/dev</code> | The directory containing files corresponding to                                                                                                                     |

physical devices (e.g., terminals, lineprinters, and disks).

|                                 |                                                                                                                                                                                                                                                              |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/dev/console</code>       | The name of the system master terminal.                                                                                                                                                                                                                      |
| <code>/dev/tty</code>           | The name of the user's terminal.                                                                                                                                                                                                                             |
| <code>/lib</code>               | The directory containing files used by some standard commands.                                                                                                                                                                                               |
| <code>/tmp</code>               | This directory contains temporary scratch files.                                                                                                                                                                                                             |
| <code>/usr/joe/project/A</code> | A typical full pathname; this one happens to be a file named <i>A</i> in the directory named <i>project</i> belonging to the user named <i>joe</i> .                                                                                                         |
| <code>bin/x</code>              | A relative pathname; it names the file <i>x</i> in sub-directory <i>bin</i> of the current working directory. If the current directory is <i>/</i> , it names <i>/bin/x</i> . If the current directory is <i>/usr/joe</i> , it names <i>/usr/joe/bin/x</i> . |
| <code>file1</code>              | Name of an ordinary file in the current directory.                                                                                                                                                                                                           |

In the XENIX system, each user resides “in” a directory called the *current directory*. All files and directories have a *parent* directory. This directory is the one immediately above, which contains the given file or directory. The XENIX file system provides special shorthand notations for this directory and for the current directory:

- The shorthand name of the current directory. Thus *./filexxx* names the same file as *filexxx*, if such a file exists in the current directory.
- .. The shorthand name of the current directory's parent directory. The shorthand name *../..* refers to the directory that is two levels above the current directory

### 3.4.4 Special Characters

XENIX lets you use special characters to specify a sequence of names containing a common pattern:

| Character      | Matches                 |
|----------------|-------------------------|
| <code>*</code> | zero or more characters |

[ ]            any character inside the brackets

?            any single character

For example, to list the names of some similarly named files, you might type

```
ls chap*
```

This produces

```
chap1.1
chap1.2
chap1.3
...
```

The asterisk can be used anywhere in the filename, and can occur several times. An asterisk by itself matches all filenames not containing slashes or beginning with periods, so

```
cat *
```

displays all files in the current directory on your terminal screen.

The asterisk is not the only pattern-matching feature available. Suppose you want to list only chapters 1 through 4, and 9:

```
ls chap[12349]*
```

The brackets ([ and ]) mean “match any of the characters inside the brackets.” You can even abbreviate a range of consecutive letters or digits:

```
ls chap[1-49]*
```

(Note that this does *not* match forty-nine filenames, but only five.)

To indicate a range of letters, enclose them within brackets:

```
[a-z]
```

This matches any character in the range “a” through “z”.

The question mark (?) matches any single character, so

```
ls ?
```

lists all files that have single-character names, and

```
ls -l chap?.1
```

lists information about the first file of each chapter (i.e., *chap1.1*, *chap2.1*, ...).

To use the special characters (\*, ?, and [ ... ]) literally, enclose the entire argument in single quotation marks. For example, the following command will print out only files named “?” rather than all one-character filenames:

```
ls `?`
```

Pattern-matching features are discussed further in Chapter 7, “The Shell.”

## 3.5 Commands

You use commands to invoke executable programs. XENIX reads the command line that you have typed, looks for a program with the given name, and then executes the program if it finds it. Command lines may also contain arguments that specify options or files that the program may need. The next two sections discuss the command line and command syntax.

### 3.5.1 Command Line

Whether you are typing commands at a terminal, or XENIX is reading commands from a file, XENIX always reads commands from command lines. The command line is a line of characters that is read by the shell command interpreter to determine what actions to perform. This interpreter, or *shell*, reads the names of commands from the command line, finds the executable program corresponding to the name of the command, then executes that program. When the program finishes executing, the shell resumes reading the command line. Thus, when you are typing at a terminal, you are editing a line of text called the *command-line buffer* that becomes a command line only when you press RETURN. You can edit the command-line buffer with the BKSP and CTRL-U keys.

Once you press RETURN, the command-line buffer is submitted to the shell as a command line. The shell reads the command line and executes the appropriate command. If you press INTERRUPT before you press RETURN, the command-line buffer is erased.

You can enter multiple commands on a single command line provided they are separated by a semicolon (;). For example, the following command line prints out the current date and the name of the current working directory:

```
date ; pwd
```

To process a command in the background, append an ampersand (&) to the command line. This mode of execution is similar to batch processing on other systems. The main advantage to placing commands in the background is that

you can execute other commands from your terminal in the “foreground” while the background commands execute. Thus:

```
du /usr &
```

determines the disk usage in the directory */usr*, a fairly time-consuming operation, without tying up your terminal.

### 3.5.2 Syntax

The general syntax for commands is as follows:

```
command [switches] [arguments] [filenames]
```

By convention, command names are lowercase.

*Switches*, also called *options*, are optional and usually *precede* other arguments and filenames. Switches consist of a dash prefix (–) and an identifying letter. For example, the *ls* command's *–r* switch

```
ls -r
```

specifies a directory listing in reverse alphabetical order.

*Arguments* provide additional information that help the command carry out its task.

*Filenames* specify the name of files required by the command.

## 3.6 Input and Output

By default, XENIX assumes that terminal input comes from the terminal keyboard and output goes to the terminal screen. To illustrate typical command input and output, type

```
cat
```

This command now expects input from your keyboard. As input, it accepts as many lines of text as you type until you press CTRL-D as an end-of-file or end-of-transmission indicator.

For example, type

```
this is two lines  
of input  
CTRL-D
```

When you press CTRL-D, input ends and output begins. The `cat` command immediately outputs the two lines you typed and, since output is sent to the terminal screen by default, that is where the two lines are sent. Thus, the complete session will look like this on your terminal screen:

```
$ cat
this is two lines
of input
this is two lines
of input
$
```

You can redirect command input and output so that input comes from a file instead of from the terminal keyboard and output goes to a file or lineprinter instead of to the terminal screen. You can also create *pipes*, which allow the output from one command to become the input to another.

### 3.6.1 Redirection

In XENIX, a file can replace the terminal for either input or output. For example

```
ls
```

displays a list of files on your terminal screen. But if you say

```
ls >filelist
```

XENIX places the list of your files in the file *filelist* (which is created if it does not exist). If “filelist” does exist, its contents will be overwritten by this command. The symbol for output redirection, the greater-than sign (`>`), means put the output from the command into the following file, rather than display it on the terminal screen.

As another example of output redirection, you can combine several files into one by capturing the output of `cat` in a file:

```
cat f1 f2 f3 >temp
```

The output append symbol (`>>`) operates very much like the output redirection symbol, except that it means “add to the end of.” So,

```
cat file1 file2 file3 >>temp
```

means concatenate *file1*, *file2*, and *file3* to the end of whatever is already in *temp*, instead of overwriting and destroying the existing contents. As with normal output redirection, if *temp* doesn't exist, XENIX creates it.

In a similar way, the input redirection symbol (<) means take the input for a program from the following file, instead of from the terminal. Thus, you could make a script of editing commands and put them into a file called *script*. Then you could execute the commands in the script on a file by typing

```
ed file <script
```

As another example, you could use **ed** to prepare a letter in the file *letter.txt*, then send it to several people with

```
mail adam eve mary joe <letter.txt
```

### 3.6.2 Pipes

One of the major innovations of the XENIX system is the concept of a *pipe*. A pipe lets you connect the output of one command to the input of another, so that the two run as a sequence of commands called a *pipeline*.

For example

```
sort frank.txt george.txt hank.txt
```

combines the three files named *frank.txt*, *george.txt*, and *hank.txt*, then sorts the output. Suppose that you want to then find all unique words in these files and view the result. You could type

```
sort frank.txt george.txt hank.txt >temp1
uniq <temp1 >temp2
more temp2
rm temp1 temp2
```

But this is more work than is necessary. What you want is to take the output of **sort** and connect it to the input of **uniq**, then take the output of **uniq** and connect it to **more**. You could use the following pipe:

```
sort frank.txt george.txt hank.txt | uniq | more
```

The vertical bar character (|) used between the **sort** and **uniq** commands indicates that the output from **sort**, which would normally have been sent to the terminal, is to be redirected from the terminal to the standard input of the **uniq** command, which in turn sends its output to the **more** command for viewing.



# Chapter 4

## Tasks

---

- 4.1 Introduction 4-1
- 4.2 Gaining Access to the System 4-1
  - 4.2.1 Logging In 4-1
  - 4.2.2 Logging Out 4-2
  - 4.2.3 Changing Your Password 4-2
- 4.3 Configuring Your Terminal 4-3
  - 4.3.1 Changing Terminals 4-3
  - 4.3.2 Setting Terminal Options 4-4
- 4.4 Editing the Command Line 4-4
  - 4.4.1 Entering a Command Line 4-4
  - 4.4.2 Erasing a Command Line 4-4
  - 4.4.3 Halting Screen Output 4-4
- 4.5 Manipulating Files 4-4
  - 4.5.1 Creating a File 4-4
  - 4.5.2 Displaying File Contents 4-5
  - 4.5.3 Combining Files 4-6
  - 4.5.4 Moving a File 4-7
  - 4.5.5 Renaming a File 4-7
  - 4.5.6 Copying a File 4-7
  - 4.5.7 Deleting a File 4-8
  - 4.5.8 Finding Files 4-8
  - 4.5.9 Linking Files 4-9
- 4.6 Manipulating Directories 4-10
  - 4.6.1 Printing the Name of Your Working Directory 4-10
  - 4.6.2 Listing Directory Contents 4-10
  - 4.6.3 Creating a Directory 4-12
  - 4.6.4 Removing a Directory 4-12

- 4.6.5 Renaming a Directory 4-12
- 4.6.6 Copying a Directory 4-12
- 4.7 Moving in the File System 4-13
  - 4.7.1 Finding Out Where You Are 4-13
  - 4.7.2 Changing Your Working Directory 4-13
- 4.8 Using File and Directory Permissions 4-14
  - 4.8.1 Changing Permissions 4-16
  - 4.8.2 Changing Directory Search Permissions 4-17
- 4.9 Processing Information 4-18
  - 4.9.1 Comparing Files 4-18
  - 4.9.2 Echoing Arguments 4-18
  - 4.9.3 Sorting a File 4-19
  - 4.9.4 Searching for a Pattern in a File 4-19
  - 4.9.5 Counting Words, Lines, and Characters 4-20
  - 4.9.6 Delaying a Process 4-21
- 4.10 Controlling Processes 4-22
  - 4.10.1 Placing a Process in the Background 4-22
  - 4.10.2 Killing a Process 4-23
- 4.11 Getting Status Information 4-24
  - 4.11.1 Finding Out Who is on the System 4-24
  - 4.11.2 Finding Out What Processes Are Running 4-24
  - 4.11.3 Finding Out Lineprinter Information 4-25
- 4.12 Using the Lineprinter 4-26
  - 4.12.1 Printing Files: lp 4-26
  - 4.12.2 Using lp Options 4-27
  - 4.12.3 Canceling a Print Request: cancel 4-27
  - 4.12.4 Finding Out the Status of Your Print Request: lpstat 4-28
- 4.13 Communicating with Other Users 4-29
  - 4.13.1 Sending Mail 4-29
  - 4.13.2 Receiving Mail 4-29
  - 4.13.3 Writing to a Terminal 4-29

- 4.14 Using the System Clock and Calendar 4-30
  - 4.14.1 Finding Out the Date and Time 4-30
  - 4.14.2 Displaying a Calendar 4-30
- 4.15 Using the Automatic Reminder Service 4-31
- 4.16 Using Another User's Account 4-31
- 4.17 Calculating 4-31



## 4.1 Introduction

This chapter explains how to perform common tasks on XENIX. The individual commands used to perform these tasks are discussed more thoroughly in the *XENIX Reference Manual*.

## 4.2 Gaining Access to the System

To use the XENIX system, you must first gain access to it by logging in. When you log in you are placed in your own personal working area. Logging in, changing your password, and logging out are described below.

### 4.2.1 Logging In

Before you can log in to the system, you must be given a system account. Your name must be added to the user list, and you must be given a password and a mailbox.

Depending on how your system is administered, you may have to add your name to the user list yourself, or someone else may be assigned this task. If you must add your own account to the system, see the *XENIX Operations Guide* and *mkuser(C)* in the *XENIX Reference Manual* for more information. This section assumes your account has already been set up.

Normally, the system sits idle and the prompt “login:” appears on the terminal screen. If your screen is blank, or displays nonsense characters, press the INTERRUPT key a few times.

When the “login:” prompt appears, follow these steps:

1. Type your login name and press RETURN. If you make a mistake, press CTRL-U to start the line again. After you press RETURN the word “Password:” appears on your screen.
2. Type your password carefully, then press RETURN. The letters do not appear on your screen as you type, and the cursor does not move. If you make a mistake, press RETURN to restart the login procedure.

If you have typed your login name and password correctly the prompt character appears on the screen. This is usually a dollar sign (\$). The prompt tells you that the XENIX system is ready to accept commands from the keyboard.

If you make a mistake, the system displays the message

```
Login incorrect  
login:
```

If you receive this message, repeat the login procedure. You must type all the letters of your user name and password correctly before you are given access to the system; XENIX does not allow you to correct mistakes during the login procedure.

When you log in you may see a message that says something like “Welcome to XENIX”, or an announcement that is of interest to all users.

### 4.2.2 Logging Out

The logout procedure is simple—all you need to do is move the cursor to a new line (one way is to press RETURN), and press

**CTRL-D**

In general, CTRL-D signifies the end-of-file in XENIX, and is often used within programs to signal the end of input from the keyboard. In such cases, CTRL-D will *not* log you out; it will simply terminate input to a particular program if you are within that program. This means that it may sometimes be necessary to press CTRL-D several times before you can log yourself out. For example, if you are in the **mail** program you must press CTRL-D once to exit the mail program, then again to log out.

### 4.2.3 Changing Your Password

To prevent unauthorized users from gaining access to the system, each authorized user must have a password. When you are first given an account on a XENIX system you are assigned a password by the system administrator. Some XENIX systems require that you change your password at regular intervals. Whether yours does or not, it is a good idea to change your password regularly to maintain system security.

Use the **passwd** command to change your password:

1. Type

**passwd**

and press RETURN. The following message appears:

```
Changing password for user:  
Old password:
```

2. Carefully type your old password.

Your password doesn't appear on the screen. If you make a mistake, press RETURN. The message “Sorry” appears, followed by the system

prompt. Begin again with Step 1.

When you have typed your old password, the message

```
New password:
```

appears.

3. Type in your new password and press RETURN.

The message

```
Retype new password:
```

appears.

4. Type your new password again.

If you make a mistake, press RETURN. The message

```
Mismatch -- password unchanged
```

appears, and you must begin again with Step 1. When you have completed the procedure, the system prompt appears.

## 4.3 Configuring Your Terminal

On most systems, the standard console terminal is already configured for use with XENIX. However, other terminals of various types may be connected to a XENIX system. In these cases it is important to know how to set terminal options and how to specify the terminal you are using. You may also want to change the configuration of the standard console terminal. The following section discusses these topics.

### 4.3.1 Changing Terminals

If you log in to XENIX on a terminal of a type different than the terminal you normally use, you may need to change the shell TERM variable. This is normally set to the proper terminal when you log in, but if you switch terminal types you must reset the TERM variable. To reset this variable, type the following line at command level:

```
TERM=termname; export TERM
```

where *termname* is the name of a known terminal. A list of known terminals is described in *terminals(M)* in the *XENIX Reference Manual*. A variety of terminals are supported; terminal capabilities are listed in the system file

*/etc/termcap.*

### 4.3.2 Setting Terminal Options

You can set a number of terminal options with the **stty** command. When entered without parameters, **stty** displays the current terminal settings. For example, typical output might look like this:

```
speed 9600 baud
erase '^h' ; kill '^u'
even -nl
```

Each of the characteristics can be set with **stty**. For more information, see *stty(C)* in the *XENIX Reference Manual*.

## 4.4 Editing the Command Line

When you sit in front of a terminal and type commands at your keyboard, there are a number of special keys that you can use.

### 4.4.1 Entering a Command Line

To enter a command line, type the characters, then press RETURN. Once you have pressed RETURN, the computer first reads the command line and then executes it.

### 4.4.2 Erasing a Command Line

To erase the current command line, press CTRL-U.

### 4.4.3 Halting Screen Output

Contents of a long file may scroll off the screen faster than you can examine them. To temporarily halt a program's output to the terminal screen, press CTRL-S. To resume output, press CTRL-Q.

## 4.5 Manipulating Files

File manipulation (creating, displaying, combining, copying, moving, naming, and deleting files) is one of an operating system's most important capabilities. The XENIX commands that perform these functions are described in the following sections.

### 4.5.1 Creating a File

To create a file and place text in it, use the editor **vi**, described in Chapter 5, "Vi: A Text Editor." In general, new files are created by commands as needed.



### 4.5.2 Displaying File Contents

The **more** command displays the contents of a file one screenful at a time, and is useful for looking at a file when you don't want to make changes to it. It has the form

**more** *options filename*

You can invoke **more** with the following options:

|                    |                                                                                                                                                                                |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>+linenumber</b> | Begins the display on the designated line in the file.                                                                                                                         |
| <b>+/text</b>      | Begins the display two lines before <i>text</i> , where <i>text</i> is a word or number. If <i>text</i> is two or more words, they must be enclosed in double quotation marks. |
| <b>-c</b>          | Redraws the screen instead of scrolling.                                                                                                                                       |
| <b>-r</b>          | Displays control characters, which are normally ignored by <b>more</b> .                                                                                                       |

To begin looking at the file *memo* at the first occurrence of the words “net gain”, for example, type

**more +/"net gain" memo**

If the file is more than one screenful long, the percentage of the file that remains is displayed on the bottom line of the screen. To look at more of the file, use the following scrolling commands:

|               |                              |
|---------------|------------------------------|
| <b>RETURN</b> | Scrolls down one line        |
| <b>d</b>      | Scrolls down one-half screen |
| <b>SPACE</b>  | Scrolls down a full screen   |
| <b>nSPACE</b> | Scrolls down <i>n</i> lines  |
| <b>.</b>      | Repeats the previous command |

You cannot scroll backward, toward the beginning of the file.

You can search forward for patterns in **more** with the slash (/) command. For example, to search for the pattern “net gain”, type

**/net gain**

and press RETURN. **More** displays the message

skipping...

at the top of the screen, and scrolls to a location two lines above “net gain”.

If you are looking at a file with **more** and decide you want to change the file, you can invoke the **vi** editor by pressing

**v**

See Chapter 5, “Vi: A Text Editor,” for information on using **vi**.

**More** quits automatically when it reaches the end of a file. To exit **more** before the end of a file, type

**q**

The **head** and **tail** commands display the first and last ten lines of a file, respectively. They are useful for checking the contents of a particular file.

For example, to look at the first ten lines of the file *memo*, type

**head memo**

You can also specify how many lines the **head** and **tail** commands display. For example

**tail -4 memo**

displays the last four lines of *memo*.

The **cat** command also displays the contents of a file. **Cat** scrolls the file until you press CTRL-S to stop it. Pressing CTRL-Q will continue the scrolling. **Cat** stops automatically at the end of a file. To stop the display before the end of the file, press INTERRUPT. To display the contents of one file, type

**cat file1**

To display the contents of more than one file, type

**cat file1 file2 file3**

### 4.5.3 Combining Files

You can use the **cat** command to combine multiple files into one new file. To combine two files named *file1* and *file2* into a new file named *bigfile*, type

```
cat file1 file2 >bigfile
```

The greater than sign (`>`) *redirects* output of the **cat** command to the new file. The original files remain intact.

You can also use **cat** to append one file to the end of another. For example, to append *file1* to *file2*, type

```
cat file1 >>file2
```

*File1* still exists as a separate entity.

#### 4.5.4 Moving a File

The **mv** command moves a file into another file in the same directory, or into another directory. For instance, to move a file named *text* to a new file named *book*, type

```
mv text book
```

To move a file into another directory, give the name of the destination directory as the final name in the **mv** command. For instance, to move *file1* and *file2* into the directory named */tmp*, type

```
mv file1 file2 /tmp
```

The two files no longer exist in your working directory, but now exist in the directory */tmp*. The above command has exactly the same effect as typing the following two commands:

```
mv file1 /tmp/file1
mv file2 /tmp/file2
```

If the last argument is the name of a directory, all files designated by filename arguments are moved into that directory.

#### 4.5.5 Renaming a File

To rename a file, simply move it to a file with the new name: the old filename is removed:

```
mv oldname newname
```

#### 4.5.6 Copying a File

There are two forms of the **cp** command: one in which files are copied into a directory, and another in which a file is copied to another file. Thus, to copy

three files into a directory named *filedir*, type

```
cp file1 file2 file3 filedir
```

In the above command, three files are copied into the directory *filedir*; the original versions still reside in the working directory. Note that the filenames are identical in the two directories. If the last argument is the name of a directory, all files designated by filename arguments are copied into that directory.

To create two copies of a file in your own working directory, you must rename the copy:

```
cp file filecopy
```

Two files with identical contents reside in the working directory.

To learn how to copy directories, see Section 4.6.7.

#### 4.5.7 Deleting a File

To delete or remove files from your working directory, type

```
rm file1 file2
```

To remove files interactively, use a command of the form

```
rm -i file1 file2
```

The system will ask you to confirm your intentions:

- Type *y* RETURN to remove the indicated file
- Type *n* RETURN to retain the indicated file

#### 4.5.8 Finding Files

The **find** command searches for files that have a specified name. **Find** is useful for locating files that have the same name, or just for finding a file if you have forgotten which directory it is in. The command has the form

```
find pathname -name filename -print
```

*Pathname* is the pathname of the directory you want to search. **Find** searches recursively; that is, it starts at the named directory and searches through all files and subdirectories under the directory specified in *pathname*.

The **-name** option indicates that you are searching for files that have a specific *filename*. For other search options you can use with **find**, see *find(C)* in the

XENIX *Reference Manual*.

*Filename* is the name of the file you are searching for.

The **-print** option indicates that you want to print the pathnames of all files that match *filename* on your terminal screen. To direct this output to a file instead of your screen, use the output redirection symbol, **>**.

For example, the following command finds every file named *memo* in the directory */usr/joe* and all its subdirectories:

```
find /usr/joe -name memo -print
```

The output might look like this:

```
/usr/joe/memo
/usr/joe/accounts/memo
/usr/joe/meetings/memo
/usr/joe/mail/memo
```

#### 4.5.9 Linking Files

The **ln** command links two files so that changes to one file are reflected in both. This can be useful if several users need to share information, or if you want a file to appear in more than one directory. This command has the form

```
ln file newfile
```

where *file* is the original file, and *newfile* is the new, linked file.

For example, the following command links *memos* in */usr/joe* to *joememos* in */usr/mary*:

```
ln /usr/joe/memos /usr/mary/joememos
```

Whenever */usr/joe/memos* is updated, the file */usr/mary/joememos* is also changed.

There are three things to remember about linking files:

1. Linking large sets of files to other parallel files can save a considerable amount of disk space.
2. Linking files that are used by more than one person is risky, because any party can alter the file and thus affect the contents of all files linked to it.
3. Removing a file from a directory does not remove other links to the file.

Thus the file is not truly deleted from the system. For example, if you delete a file that has 4 links, 3 links remain.

For more information about linking, see *ln(C)* in the *XENIX Reference Manual*.

## 4.6 Manipulating Directories

Because of the hierarchical organization of the file system, there are many directories and subdirectories in the XENIX system. Within the file system are directories for each user of the system. Within your user directory you can create, delete, and copy directories. Commands that let you manipulate directories are described in the following sections.

### 4.6.1 Printing the Name of Your Working Directory

All commands are executed relative to a “working” directory. The name of this directory is given by the **pwd** command, which stands for “print working directory.” For instance, if your current working directory is */usr/joe*, when you type:

```
pwd
```

you will receive the output

```
/usr/joe
```

You should always think of yourself as residing “in” your working directory.

### 4.6.2 Listing Directory Contents

You can list the contents of a directory with the **ls** command. This command sorts the names of files and directories in a given directory, and lists them in columns. If no directory name is given, **ls** lists the contents of the current directory. The **ls** command has the form

```
ls options name
```

For example, to list the contents of the directory *work*, type

```
ls work
```

Your output might look like this:

|          |          |       |
|----------|----------|-------|
| accounts | meetings | notes |
| mail     | memos    | todo  |

If no *name* is specified, **ls** lists the contents of the current directory.

The following options control the sort order and the information displayed by the `ls` command:

- a** Lists all files in the directory, including the “hidden” files (filenames that begin with a dot, such as *.profile* and *.mailrc*).
- r** Lists names in reverse alphabetical order.
- t** Lists names in order of last modification, beginning with the latest (most recently modified). When used with the **-r** option, lists the oldest first.
- F** Marks directories with a backslash(\) and executable files with an asterisk (\*).
- R** Lists all files and directories in the current directory, plus each file and directory *below* the current one. The “R” stands for “recursive”.

The **-l** option displays a “long” directory listing, producing output that might look something like this:

```
total 501
drwxr-x--- 2 boris  grp1  272 Apr  5 14:33 dir1
drwxr-x--- 2 enid   grp1  272 Apr  5 14:33 dir2
drwxr-x--- 2 iris   grp1  592 Apr  6 11:12 dir3
-rw-r----- 1 olaf   grp2  282 Apr  7 15:11 file1
-rw-r----- 1 olaf   grp2   72 Apr  7 13:50 file2
-rw-r----- 1 olaf   grp2 1403 Apr  1 13:22 file3
```

From left to right, the information given for each file or directory includes

- Permissions
- Number of links
- Owner
- Group
- Size in bytes
- Time of last modification
- Filename

Section 4.8 discusses how to use this additional directory information.

For more information about listing the contents of a directory, see *ls(C)* in the *XENIX Reference Manual*.

### **4.6.3 Creating a Directory**

To create a subdirectory in your working directory, use the **mkdir** command. For example, to create a new directory named *phonenumbers*, type

```
mkdir phonenumbers
```

### **4.6.4 Removing a Directory**

To remove a directory from your working directory, use the **rmdir** command. For example, to remove the directory named *phonenumbers* from the current directory, type

```
rmdir phonenumbers
```

A directory must be *empty* before it can be removed; this prevents accidental deletions of files and directories.

### **4.6.5 Renaming a Directory**

To rename a directory, use the **mv** command. For example, to rename the directory *little.dir* to *big.dir*, type

```
mv little.dir big.dir
```

This is a simple renaming operation; no files are moved.

### **4.6.6 Copying a Directory**

The **copy** command copies directories. This command has the form

```
copy options olddir newdir
```

The **copy** command has the following options:

- l**       Links the copied files to the original
- m**       Gives the copied files the same modification dates as the original files
- r**       Copies the directory recursively, i.e., copies all the directories under the named directory

To copy all the files in the directory */usr/joe/memos* into */usr/joe/notes*, type



`copy /usr/joe/memos /usr/joe/notes`

## 4.7 Moving in the File System

Each directory in the XENIX system should be thought of as a place that you can move into or out of. This place is called either your working directory or current directory. The commands used to find out where you are and to move around in the structure are discussed below.

### 4.7.1 Finding Out Where You Are

Your current location in the file system is the name of the working directory. You can find out this name by using the **pwd** command, which stands for “print working directory”. For example, if you are in the directory */usr*, then typing the command

**pwd**

prints out the name

*/usr*

### 4.7.2 Changing Your Working Directory

Your working directory represents your location in the file system: it is “where you are” in XENIX. To alter this location in the XENIX file system, use the change directory (**cd**) command:

**cd**

This changes your working directory to your home directory. To move to any other directory, specify that directory as an argument to **cd**, as in the following command:

**cd /usr**

To move up one directory from your current directory, type

**cd ..**

For example, the above command would move you from the directory */usr/joe/work* to */usr/joe*. Similarly, the command

**cd ../..**

would move you from the directory */usr/joe/work* to */usr*, moving you up *two*

directories.

## 4.8 Using File and Directory Permissions

The XENIX system allows the owner to restrict access to files and directories, limiting who can read, write and execute files owned by him. To determine the permissions associated with a given file or directory, use the `ls -l` command. The output from the `ls -l` command should look something like this:

```
total 501
drwxr-x--- 2 boris  grp1   272 Apr  5 14:33 dir1
drwxr-x--- 2 enid   grp1   272 Apr  5 14:33 dir2
drwxr-x--- 2 iris   grp1   592 Apr  6 11:12 dir3
-rw-r----- 1 olaf   grp2   282 Apr  7 15:11 file1
-rw-r----- 1 olaf   grp2    72 Apr  7 13:50 file2
-rw-r----- 1 olaf   grp2  1403 Apr  1 13:22 file3
```

Permissions are indicated by the first ten characters of the output. The permissions for *dir1*, the first file in the above list, are

`drwxr-x---`

The first character indicates the type of file and must be one of the following:

| Character | Type of File                                               |
|-----------|------------------------------------------------------------|
| -         | ordinary file                                              |
| d         | directory                                                  |
| c         | character special device such as a lineprinter or terminal |
| b         | block special device such as a hard or floppy disk         |
| s         | XENIX 3.0 semaphore                                        |
| m         | XENIX 3.0 shared data file                                 |
| p         | named pipe                                                 |

The next nine characters represent three sets of three permissions:

- Owner permissions
- Group permissions
- All other user permissions

The three characters within each set indicate permission to read, to write, and to execute the file as a command, respectively. (For a directory, “execute” permission means permission to search the directory for any included files or directories.)

Ordinary file permissions have the following meanings:

|   |                                         |
|---|-----------------------------------------|
| r | The file is readable                    |
| w | The file is writeable                   |
| x | The file is executable                  |
| - | The indicated permission is not granted |

Directory permissions have the following meanings:

|   |                                                                                                                                                                                                                                                                                                       |
|---|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| r | Files can be listed in the directory; the directory must also have “x” permission.                                                                                                                                                                                                                    |
| w | Files can be created or deleted in the directory; as with “r,” the directory itself must also have “x” permission.                                                                                                                                                                                    |
| x | The directory can be searched. A directory must have “x” permission before you can move to it with the <code>cd</code> command (i.e., <code>cd</code> to it), access a file within it, or list the files in it. Remember that a user must have “x” permission to do anything useful to the directory. |

The following are some typical directory permission combinations:

|            |                                                                                                                                                                                                                                                                                                               |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| d-----     | No access at all. This is the mode that denies access to the directory to a class of users.                                                                                                                                                                                                                   |
| drwx-----  | Allows access by the owner to use <code>lc</code> , create files, delete files, access files (subject to file permissions), and <code>cd</code> to the directory. This is the typical permission for the owner of a directory.                                                                                |
| drwxr-x--- | Allows access by members of the group to use <code>lc</code> , and access files subject to file permissions. Group members can <code>cd</code> to this directory, but cannot create or delete files in it. This is the typical permission an owner gives to others who need access to files in his directory. |
| drwx--x--x | With these permission settings users other than the owner cannot use <code>lc</code> but can <code>cd</code> to the directory. Other                                                                                                                                                                          |

users can only access a file within this directory by its exact name; they cannot use special characters. Files cannot be created or deleted in the directory by anyone except the owner. This mode is rarely used, but can be useful if you want to give someone access to a specific file in a directory without permitting access to other files in the same directory.

This section discusses ordinary files, executable files, and directories only. For information about other types of files, see *ls(C)* in the *XENIX Reference Manual*.

### 4.8.1 Changing Permissions

The **chmod** command changes the read, write, execute, and search permissions of a file or directory. This command is useful if you have created a file in one mode, but want to give others permission to read, write or execute it. The **chmod** command has the form

**chmod** *instruction filename*

The *instruction* segment of the command indicates which permissions you want to change for which class of users. There are four classes of users:

|   |                                                   |
|---|---------------------------------------------------|
| u | User, the owner of the file or directory          |
| g | Group, the group the owner of the file belongs to |
| o | Other, all users of the system                    |
| a | All classes of users                              |

There are three types of permissions:

|   |                                                                                   |
|---|-----------------------------------------------------------------------------------|
| r | Read, which allows permitted users to look at, but not change or delete the file. |
| w | Write, which allows permitted users to change or even delete the file.            |
| x | Execute, which allows permitted users to execute the file as a command.           |

For example, assume *file1* exists with the following permissions:

```
-rw-r----
```

The owner of the file has read and write permission, group members have read

permission, and others have no access at all.

To give *file1* read permission for *all* classes of users, type

```
chmod a+r file1
```

In the instruction segment of the command (a+r) the “a” stands for “all.” The resulting permissions are

```
-rw-r--r--
```

If *file1* had the attributes

```
-rw-----
```

the following command would give write and execute permissions to members of a group only:

```
chmod g+wx file1
```

The permission attributes would then look like this:

```
-rw--wx---
```

To remove write and execute permission by the user (owner) and group associated with *file1*, type

```
chmod ug-wx file1
```

#### 4.8.2 Changing Directory Search Permissions

Directories also have an execute permission. This attribute signifies search permission, rather than execute permission, since directories cannot be executed. If this permission is denied to a particular user, then that user cannot even list the names of the files in the directory.

For example, assume that the directory *dir1* has the following attributes:

```
drwxr-xr-x
```

To prevent users from examining *dir1*, type

```
chmod o-xr dir1
```

to remove search permission.

The new attributes for *dir1* are

drwxr-x---

## 4.9 Processing Information

In many cases, files will contain information that you may want to process. Various utility programs exist on XENIX to process information. A set of these programs and their uses are described in the following sections.

### 4.9.1 Comparing Files

The **diff** command prints out those lines that differ between two specified files.

For example, suppose a file named *men* has the contents

```
Now is the time for all good men to  
Come to the aid of their party.
```

and a file named *women* has the contents

```
Now is the time for all good women to  
Come to the aid of their party.
```

the command

```
diff men women
```

would produce the following result:

```
1c1  
< Now is the time for all good men to  
---  
> Now is the time for all good women to
```

You can use the **diff3** command to compare three files. For information about **diff3** see *diff3(C)* in the *XENIX Reference Manual*.

### 4.9.2 Echoing Arguments

The **echo** command echos arguments to the standard output device. For example, typing

```
echo hello
```

outputs

```
hello
```

on the terminal screen. To output several lines of text, enclose the echoed argument in double quotation marks and press RETURN between lines. A secondary prompt will appear until you type the final double quotation mark. For example, type

```
echo "Now is the time
For all good men
To come to the
Aid of their party."
```

This will output

```
Now is the time
For all good men
To come to the
Aid of their party.
```

**Echo** is particularly useful when you program in the shell command language. For more information about the shell, see Chapter 7, "The Shell."

### 4.9.3 Sorting a File

The **sort** command sorts the lines of a file according to the ASCII collating sequence (i.e., it alphabetizes them).

For example, to sort a file and see the results on screen, type

```
sort filename
```

You can redirect the results of a sort to a new file:

```
sort phonelist > phonesort
```

**Sort** is useful for sorting the output from other commands. For example, to sort the output from execution of a **who** command, type

```
who | sort > whosort
```

This command takes the output from **who**, sorts it, and then sends the sorted output to the file *whosort*.

A wide variety of options are available for **sort**. For more information, see *sort(C)* in the *XENIX Reference Manual*.

### 4.9.4 Searching for a Pattern in a File

The **grep** command selects and extracts lines from a file, printing only those lines that match a given pattern.

For example, to print out all lines containing the word “tty38”, type

```
grep 'tty38' filename
```

You should always enclose the pattern you are searching for in single quotation marks (’), so that special metacharacters are not expanded unexpectedly by the shell.

As another example, assume that each line of a file named *phonelist* contains a name followed by a phone number. Assume also that there are several thousand lines in this list. You can use **grep** to find the phone number of someone named Joe, whose phone number prefix is 822:

```
grep 'joe' phonelist | grep '822-'>joes.number
```

**Grep** finds all occurrences of lines containing the word “joe” in the file *phonelist*. The output from this command is then filtered through another **grep** command, which searches for an “822-” prefix, thus removing any unwanted joes. Finally, assuming that a unique phone number for joe exists with the “822-” prefix, that name and number are placed in the file *joes.number*.

For more information about **grep**, its relatives **fgrep** and **egrep**, and the types of patterns it can be used to search for, see *grep(C)* in the *XENIX Reference Manual*.

#### 4.9.5 Counting Words, Lines, and Characters

**Wc** is a utility for counting words in a file. The letters “wc” stand for word count. Words are presumed to be separated by punctuation, spaces, tabs, or newlines. **Wc** also counts characters and lines; all three counts are reported by default. For example, to count the number of lines, words, and characters in the file *textfile*, type

```
wc textfile
```

Typical output describing lines, words and characters might be

```
4432    18188   97808   textfile
```

To specify a count of characters, words, or lines only, you must use an appropriate mnemonic switch. To illustrate, examine the following three commands and the output produced by each:



```

wc -c textfile
    97808 textfile

```

```

wc -w textfile
    18188 textfile

```

```

wc -l textfile
    4432 textfile

```

The first example prints out the number of characters in *textfile*, the second prints out the number of words, and the third prints out the number of lines.

#### 4.9.6 Delaying a Process

The **at** program lets you set up commands to be executed at a specified time. It is useful if you want to execute a command when you are not planning to be at your terminal, or even logged in.

The **at** command has the form

```
at time day file
```

*Time* is the time of day, in digits, followed by “am” or “pm”. One- and two-digit numbers are interpreted as hours, three- and four-digit numbers as hours and minutes. More than four digits are not permitted.

*Day* is optional. It is either a month name followed by a day number, or a day of the week. If no day is specified, the command will be executed today.

*File* is the name of the file that contains the command or commands to be executed.

For example, to find out what processes are running at 10 pm on Tuesday, place the following line in a file named *use*:

```
ps -a>/usr/myname/use
```

(See Chapter 5, “Vi: A Text Editor,” for information on creating and inserting text into files.)

After you have written out the file and returned to command level, type

```
at 10pm tues use
```

Press RETURN. The XENIX prompt reappears and you may continue working. At 10 pm on Tuesday, XENIX will execute **ps** and place the output in the file *use*. **At** is unaffected by logging out.

To check what files you have waiting to be processed, use the **at -l** command. **At -l** lists the files to be processed, along with the following information:

- The file's user ID
- The file's ID number
- The date and time the file will be processed

To cancel an **at** command, first check the list of files to be processed and note the file ID number. Then use the **at -r** command to remove the file or files from the list. The **at -r** command has the form

**at -r** *number*

For example

**at -r** 84.032.2300.21

removes file number 84.032.2300.21, canceling whatever commands were included in that file. A user can remove only his own files.

## 4.10 Controlling Processes

In XENIX, several processes can run at the same time. For example, you may run the **sort** program on a file in the *background*, and edit another file in the *foreground* while the **sort** program is running. Processes you directly control from your keyboard are called foreground processes. Other processes, which you can initiate but otherwise have little control over, are called background processes. At any one time you can have only one foreground process executing, but multiple background processes may execute simultaneously. This section describes procedures for controlling foreground and background processes.

### 4.10.1 Placing a Process in the Background

Normally, commands sent from the keyboard are executed in strict sequence; one command must finish executing before the next can begin. A background process, in contrast, need not finish executing before you give your next command. Background processing is especially useful for commands that may take a long time to complete.

To place a process in the background, type an ampersand (&) at the end of the command. For example, to count the number of words in several large files while simultaneously continuing with whatever else you have to do, type

```
wc file1 file2 file3 >count&
```

Output is collected in the file *count*. If output were not put in *count*, it would appear on the screen at unpredictable times as you continue with your work.

When processes are placed in the background, you lose control of them as they execute. For instance, typing INTERRUPT does *not* stop a background process. You must use the **kill** command to terminate a background process.

#### 4.10.2 Killing a Process

To stop execution of a foreground process, press your terminal's INTERRUPT key. This kills whatever foreground command is currently running. To kill all processes executing in the background, type

```
kill 0
```

To kill only a specified process executing in the background, first type

```
ps
```

**Ps** displays the Process Identification Numbers (PIDs) of your existing processes, for example

| PID  | TTY | TIME | CMD          |
|------|-----|------|--------------|
| 3459 | 03  | 0:15 | -sh          |
| 4831 | 03  | 1:52 | cc program.s |
| 5185 | 03  | 0:00 | ps           |

In the above example, you might type

```
kill 4831
```

where *4831* is the PID of the process that you want killed.

---

#### Note

Killing a process associated with the **vi** editor may leave the terminal in a strange mode. Also, temporary files that are normally created when a command starts, and then deleted when the command finishes, may be left behind after a **kill** command. Temporary files are normally kept in the directory */tmp*. This directory should be checked periodically and old files deleted.

---

## 4.11 Getting Status Information

Because XENIX is a large, self-contained computing environment, there are many things that you may want to find out about the system itself, such as who is logged in, how much disk space there is, what processes are currently running. This section explains how to obtain system information.

### 4.11.1 Finding Out Who is on the System

The **who** command lists the names, terminal line numbers, and login times of all users currently logged on to the system. For example, type

```
who
```

This command produces output of the form

```
arnold   tty02  Apr 7 10:02
daphne   tty21  Apr 7 07:47
elliott  tty23  Apr 7 14:21
ellen    tty25  Apr 7 08:36
gus      tty26  Apr 7 09:55
adrian   tty28  Apr 7 14:21
```

The **finger** command provides more detailed information, such as office numbers and phone extensions. For more information, about using **finger** see *finger(C)* in the *XENIX Reference Manual*.

### 4.11.2 Finding Out What Processes Are Running

Because commands can be processed in the background, it is not always obvious which processes you are responsible for. The **ps** command stands for *process status* and displays information about processes associated with your terminal. The **ps** command produces output of the form

| PID   | TTY | TIME | CMD           |
|-------|-----|------|---------------|
| 10308 | 38  | 1:36 | ed chap02.man |
| 49    | 38  | 0:29 | -sh           |
| 11267 | 38  | 0:00 | ps            |

The PID column gives a unique process identification number that can be used to kill a particular process. The TTY column indicates the terminal the process is associated with. The TIME column shows the cumulative execution time for the process.

To see all the processes running on the system, use the **a** option:

**ps -a**

To find out about the processes running on another terminal, specify the terminal number. For example, to find out what processes are associated with terminal 13, type

**ps -t13**

For more information about **ps** and its options, see *ps(C)* in the *XENIX Reference Manual*.

### 4.11.3 Finding Out Lineprinter Information

The **lpstat** command displays information about files waiting to be printed.

To find out the status of one file, you need to know the *request ID*. This appears on your screen when you make print requests using the **lp** command. The request ID has the form

*printer-idnumber*

*Printer* is the name of the printer your file will be printed on (check with your system administrator for the names of printers available to you). *Idnumber* is a unique number identifying your file.

To find out the status of a particular file, type

**lpstat requestID**

**Lpstat** responds by displaying the date and time you made your print request and the number of characters remaining to be printed.

To find out the status of all your files waiting to be printed on the lineprinters, type

**lpstat**

**Lpstat** responds by displaying the request IDs and status information for all your files.

You can find out what files are waiting to be printed on a particular printer by using **lpstat** with the **-p** option. This command has the form

**lpstat -p printer**

**Lpstat** responds by printing the request IDs and status information for all the files waiting to be printed on the named printer.

For more information on **lpstat** and its options, see *lpstat(C)* in the XENIX *Reference Manual*.

## 4.12 Using the Lineprinter

The XENIX lineprinter commands are easy to use and give you great flexibility when you want to print a file. With a few simple commands, you can print multiple copies of a file, cancel a print request, or ask for a special option on a particular printer. Since the XENIX lineprinter system is designed to be easily adapted to many different environments, check with your system administrator to find out what lineprinters and printer options are available to you.

### 4.12.1 Printing Files: **lp**

To print copies of your files, you can use either the **lp** or **lpr** commands. These commands are equivalent. The examples in this section use **lp**.

For example, to print one copy of a file named *memo*, type

```
lp memo
```

To print multiple files, add the filenames to the command line:

```
lp memo report letter
```

When you make print requests, **lp** responds by displaying your request ID on your terminal screen. Request IDs are of the form

```
pr4-532
```

The first part (pr4) is the name of the printer your file will be printed on. The second part (532) identifies your file. Should you later wish to cancel your print request or check its status, you will find it useful to remember your request ID. For more information on these tasks, see Sections 4.12.3, and 4.12.4.

One copy of each file you named will be printed on the default destination printer on your system.

You can use **lp** with pipes and other commands. The command to paginate a file is **pr**. To paginate and print a file named *textfile*, type

```
pr textfile | lp
```

To sort, paginate, and print a file named *datafile*, type

```
sort datafile | pr | lp
```

### 4.12.2 Using lp Options

The **lp** command has several options to help you control the output from your printer.

You can specify the number of copies you want printed by using the number option, **-n**. For example, to print two copies of a file named *report*, type

```
lp report -n2
```

Another option, **-d**, specifies your file's destination, that is, which printer your file will be printed on. Check with your system administrator for the names of the printers available to you. To have two copies of a file named *report* printed on a printer named *quick*, type

```
lp report -n2 -dquick
```

Other useful options include

- c** Makes a copy of the files you are printing. This prevents you from inadvertently removing or changing the file before it is printed.
- m** Sends you mail telling you your file has been printed.
- o** Specifies printer options. For example, you may be able to request that your document be printed using 12 pitch type. Check with your system administrator to see what options are available for each printer or groups of printers on your system.
- r** Removes your files after printing.

For more information on options available for the **lp** command, see *lp(C)* in the *XENIX Reference Manual*.

### 4.12.3 Canceling a Print Request: cancel

You can cancel a print request. For example, to stop printing a file with a request ID of *laser-245*, type

```
cancel laser-245
```

The **cancel** command immediately stops the file from being printed, even if the printer has already begun the print request.

You can also use the **cancel** command to stop whatever is currently printing on a particular printer.

For example, to cancel whatever file is currently printing on a printer named *slow*, type

```
cancel slow
```

If the file did not belong to you, mail will automatically be sent to the file's owner reporting that the print request was canceled.

#### 4.12.4 Finding Out the Status of Your Print Request: *lpstat*

To find out the status of your files waiting to be printed, type

```
lpstat
```

You'll receive a message of the form

|           |        |      |              |
|-----------|--------|------|--------------|
| prt1-121  | chrisw | 450  | Dec 15 09:30 |
| laser-450 | chrisw | 4968 | Dec 15 09:46 |

The first column shows the request ID for each of your files being printed; the second column is your login name; the third column lists the number of characters to be printed; the fourth column lists the dates and times you made your print requests.

To learn the status of a particular file, use the *lpstat* command with your request ID. For example, to find out the status of a file with the request ID of *daisy-256*, type

```
lpstat daisy-256
```

*Lpstat* displays the status of that file only.

You can also request the status of various printers on your system by using the *-p* option or by giving the name of the particular printer you are interested in.

To find out the status of all the printers on your system, type

```
lpstat -p
```

To find out the status of a printer named *quick*, type

```
lpstat -pquick
```

*Lpstat* displays the request ID and status information for each file currently waiting to be printed on the printer named *quick*.

For more information on *lpstat* and its options, see *lpstat(C)* in the XENIX



*Reference Manual.*

## 4.13 Communicating with Other Users

Because the XENIX system supports multiple users, it is very convenient to communicate with other users of the system. The various methods of communication are described below. See Chapter 6, “Mail,” and **mail(C)** in the *XENIX Reference Manual* for more information.

### 4.13.1 Sending Mail

**Mail** is a system-wide facility that permits you and other system users to send and receive mail. To send mail to another user on the system, type

```
mail username
```

where *username* is the name of any user of the system. Following entry of the command, you enter a subject title followed by the actual text of the message you want to send. Type CTRL-D to complete your message.

### 4.13.2 Receiving Mail

When you log in you may sometimes see the message

```
you have mail
```

To read your mail, type

```
mail
```

A heading for each message appears on your terminal screen. When you press RETURN, the contents of the first message are displayed. Subsequent messages appear one message at a time, most recent message first, each time you press RETURN.

Once each message is displayed, you must decide what to do with it. The two basic responses are **d**, which deletes the message, and RETURN, which retains the message in your mailbox. To exit mail, type: **q**, for “quit.”

### 4.13.3 Writing to a Terminal

To write directly to another user’s terminal, use the **write** command. For example, to write to joe’s terminal, type

```
write joe
```

After you have executed the command by pressing RETURN, each subsequent line that you type is displayed both on your own terminal screen and on joe’s.

To terminate the writing of text to joe, enter a CTRL-D alone on a line. The procedure for a two-way write is for each party to end each message with a distinctive signal, normally (o) for “over”; when a conversation is about to be terminated use the signal (oo) for “over and out”.

## 4.14 Using the System Clock and Calendar

Several XENIX commands will tell you the date and time, or display a calendar for any month or year you choose. The following sections explain these commands.

### 4.14.1 Finding Out the Date and Time

The **date** command displays the time and date. Type

```
date
```

The date and time appear on the next line of the display.

### 4.14.2 Displaying a Calendar

The **cal** command displays the calendar of any month or year you specify. This command has the form

```
cal month year
```

For example, to display the calendar for March, 1952 type

```
cal 3 1952
```

The result is

```
      March 1952
S  M Tu  W Th  F  S
                1
2  3  4  5  6  7  8
9 10 11 12 13 14 15
16 17 18 19 20 21 22
23 24 25 26 27 28 29
30 31
```

The month must always be expressed as a digit. To display the calendar for an entire year, leave out the month. The year must always be expressed in full; the command “cal 84” displays the calendar for the year 84, not 1984.

## 4.15 Using the Automatic Reminder Service

An automatic reminder service is normally available for all XENIX users. Once each day, XENIX uses the **calendar** command to examine each user's home directory for a file named *calendar*, the contents of which might look something like this

```
1/23 David's wedding
2/9 Mira's birthday
3/30 Paul's birthday
4/27 Meeting at 2:00
9/1 Karen's birthday
10/3 License renewal
```

**Calendar** examines each line of the calendar file, extracting from the file those lines containing today's and tomorrow's dates. These lines are then mailed to you to remind you of the specified events.

## 4.16 Using Another User's Account

You can easily access another user's files, regardless of the permission settings, with the **su** command. The **su** procedure resembles logging in, and you must know the other user's password. For example, to become user Joe, type

```
su joe
```

and press RETURN. When the password prompt appears, type Joe's password. To cancel the effect of the **su** command and return to your own account, press CTRL-D.

## 4.17 Calculating

The **bc** command invokes an interactive desk calculator that can be used as if it were a hand-held calculator.

For more information, see Chapter 8, "BC: A Calculator."

